Student Result Prediction System

Detailed Project Report (DPR)

Machine Learning Web Application using Python & Streamlit

VaultSphere AI Technologies Pvt. Ltd.

Developed by: Derin Devis

June 2026

Project Title	Student Result Prediction System
Technology	Python, Streamlit, scikit-learn, Pandas, NumPy, Matplotlib, Seaborn
Domain	Machine Learning / Data Science
Model Used	Logistic Regression
Dataset	500 Synthetic Student Records
Developed By	Derin Devis
Registration Number	23F41A3314
Organization	VaultSphere AI Technologies Pvt. Ltd.
Date	June 2026

1. Abstract

Predicting student outcomes early in the academic term is highly valuable for educational institutions, as it allows instructors to identify struggling students and provide targeted support before final examinations. In this project, I developed the **Student Result Prediction System**, an interactive web application built with Python and Streamlit. The application is designed to automate the process of forecasting student academic outcomes based on individual behavioral and academic data.

The predictive engine is powered by a Logistic Regression model trained on a synthetic dataset of 500 student records. The model analyzes five key inputs: daily study hours, class attendance, previous exam performance, completed assignments, and daily sleep hours. When a user inputs these parameters, the system processes them to output a real-time prediction indicating whether the student is likely to PASS or FAIL, accompanied by a percentage confidence score.

In addition to predictions, the application features an interactive analytics dashboard and model evaluation charts. This ensures that school administrators and educators can easily interpret cohort statistics and understand the reliability of the underlying machine learning model, helping them design better student support strategies.

2. Objectives

- Design and implement a predictive classification model to identify at-risk students before final exams.
- Analyze the combined influence of academic habits (study hours, attendance) and wellness factors (sleep) on success.
- Build an intuitive, interactive web interface using Streamlit so educators can run queries without writing code.
- Provide real-time PASS / FAIL predictions backed by probability-based confidence metrics.
- Develop a cohort visualization dashboard featuring metrics, scatter plots, and histograms for grade distribution.
- Assess and present model reliability transparently using Accuracy, Precision, Recall, and F1 Score.
- Build a dynamic, downloadable PDF report generator that lets teachers export prediction sheets on demand.

3. Problem Statement

In typical school and college environments, monitoring student progress and flagging those who are struggling is done manually and subjectively. Teachers try to track attendance and marks, but this is time-consuming and prone to oversight—especially in large classes. Consequently, at-risk students are often identified only after they have failed their final exams, when it is too late to intervene.

This project solves this problem by creating a data-driven prediction tool. By modeling historical academic data and learning relationships between multiple student metrics, the system provides teachers with an automated early warning system. This allows them to identify struggling students during the semester and implement targeted tutoring programs in time.

4. Tools & Technologies Used

Tool / Library	Version	Purpose
Python	3.11+	Core programming language
Streamlit	1.35+	Frontend framework for creating interactive web layouts in Python
scikit-learn	1.4+	Machine Learning library used for scaling, splitting, and fitting the classifier
Pandas	2.0+	Structured data manipulation using DataFrames
NumPy	1.26+	Mathematical computing and synthetic dataset generation
Matplotlib	3.8+	Static chart generation (scatter plots, pie charts, histograms)
Seaborn	0.13+	Advanced data visualization, specifically the confusion matrix heatmap

5. Dataset Description

Due to privacy regulations regarding student academic files, real institutional data was not accessible for this project. To train the predictive model, I programmatically generated a synthetic dataset of **500 student records** using NumPy. This dataset simulates academic behaviors observed in typical higher education cohorts. To ensure the dataset remains realistic, the final outcomes are calculated using a weighted grading formula combined with random Gaussian noise, mimicking the natural variations found in student academic performance.

Feature	Range	Description
Hours_Study	1 – 12	Average number of daily study hours outside of lectures
Attendance	40 – 100%	Percentage of lectures attended throughout the semester
Previous_Marks	30 – 100	Grades secured by the student in the mid-term examination
Assignments_Done	0 – 10	Total number of assignments submitted (out of 10)
Sleep_Hours	4 – 10	Average daily sleep hours — representing student wellness
Result (Target)	0 / 1	Binary class label: 0 = Fail (score < 50), 1 = Pass (score >= 50)

The target label *Result* is derived from a computed final marks score that places the highest emphasis on study hours and previous marks, with attendance, completed assignments, and sleep acting as supporting indicators. A student is labeled as 'Pass' (1) if their calculated marks are 50 or above, and 'Fail' (0) if they fall below 50. This clear baseline ensures the model can learn a sensible decision boundary.

6. System Architecture

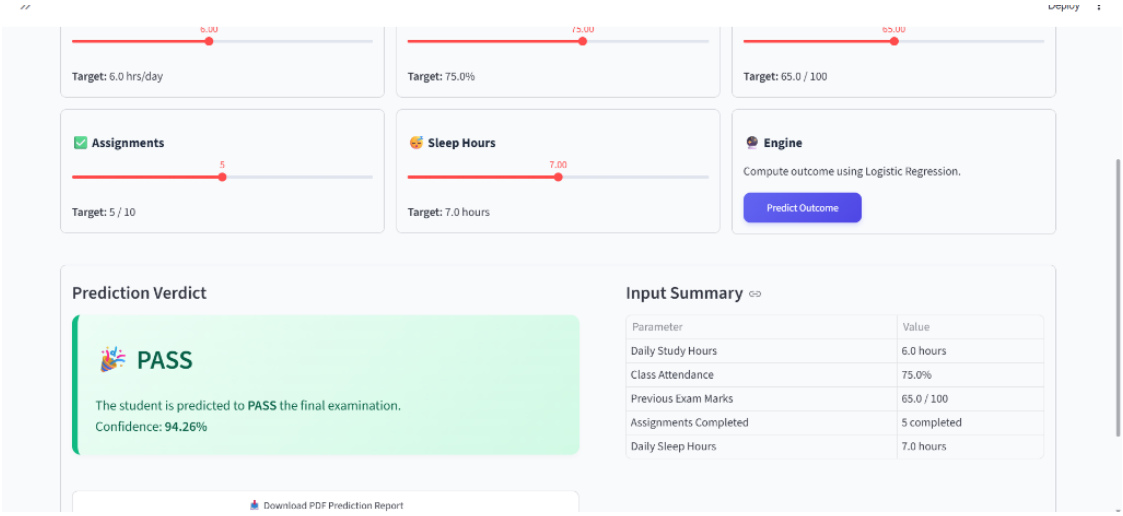
The application is structured as a modular Python pipeline that runs locally on a single machine. The backend handles data creation, standardization, and prediction using scikit-learn. The frontend utilizes Streamlit to render interactive sliders and charts, providing a seamless user experience. This design avoids complex server setups, making it easy to deploy and run.

Step	Component	Description
1	Data Generation	NumPy generates a tabular dataset of 500 records with realistic academic correlations
2	Pre-processing	Pandas organizes the features; StandardScaler normalizes values for stable fitting
3	Model Training	Dataset is split 80/20; Logistic Regression is fitted on 400 training records
4	Prediction	Slider values are scaled and passed to the model, returning result and probability
5	Visualization	Matplotlib and Seaborn generate scatter plots, pie charts, and heatmaps on demand
6	Web Interface	Streamlit acts as the presentation layer, running on http://localhost:8501

7. Application Features (3 Tabs)

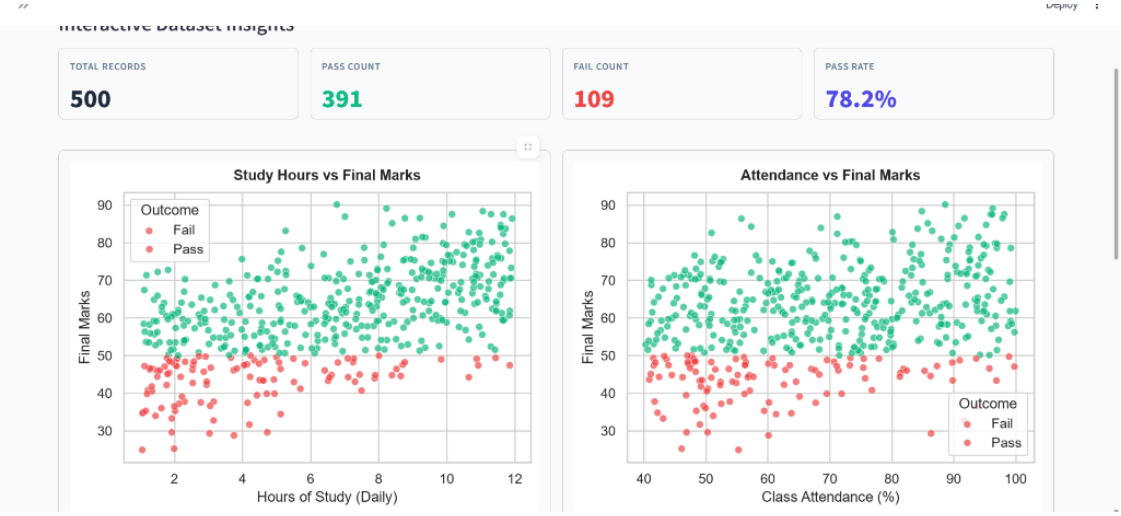
To make the predictive engine accessible to non-technical users, I organized the Streamlit interface into three clear, functional tabs:

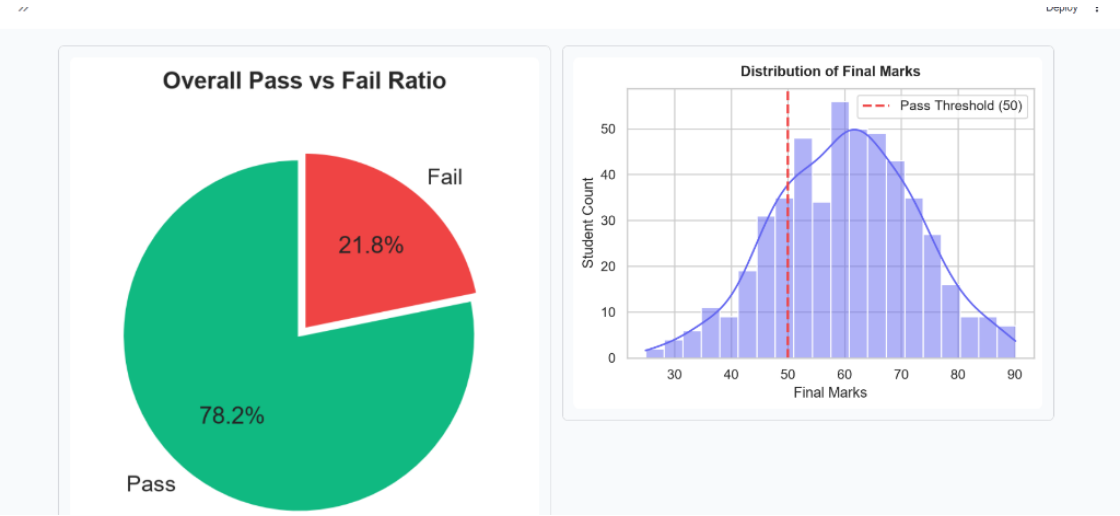
- **Tab 1 — Predict Result:** Provides interactive sliders for the five academic and lifestyle parameters. Upon clicking 'Predict Outcome', the backend scales the inputs and evaluates them using the trained model. The interface then displays a green PASS banner or a red FAIL warning with a confidence percentage, and provides a download button to export the result sheet as a formatted PDF report.



• Tab 2 — Data Analysis

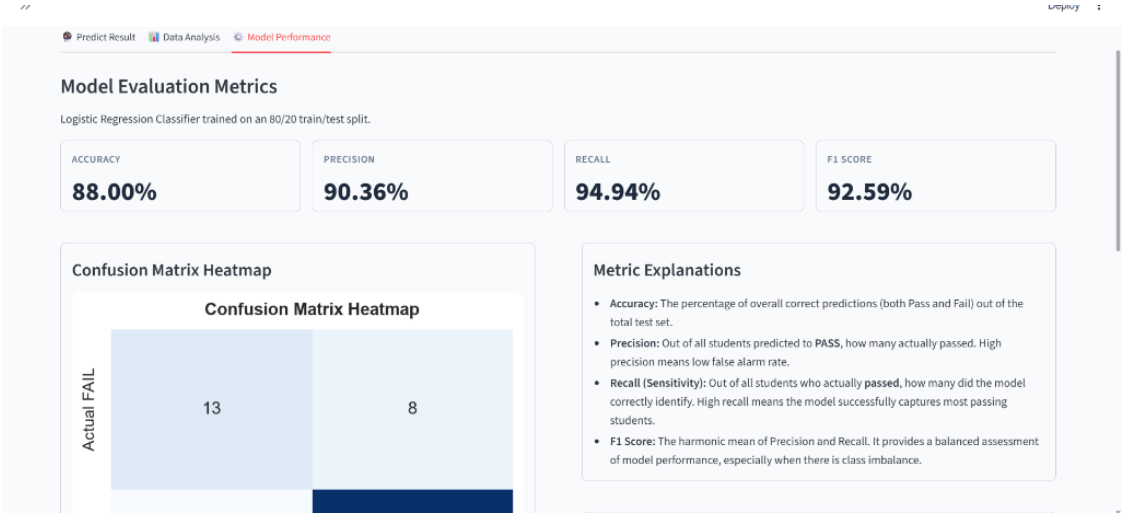
Displays cohort-wide metrics (such as the total records, average pass rate, pass count, and fail count) and renders interactive charts. These include study time vs. marks scatter plots, class attendance trends, overall pass vs. fail ratios, and grade distribution histograms, helping teachers spot general performance patterns.

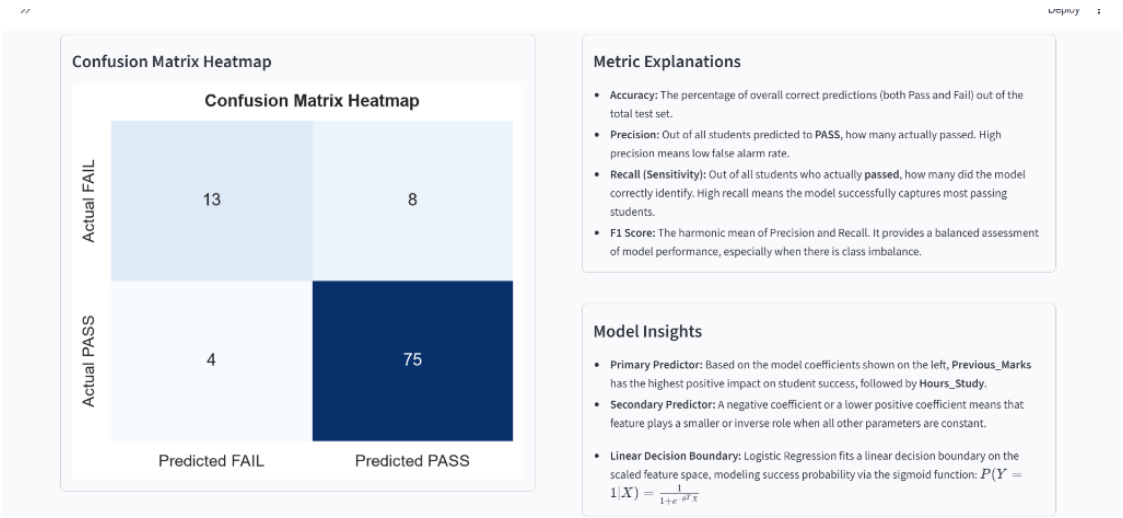




• Tab 3 — Model Performance

Evaluates the underlying classifier, showing its Accuracy, Precision, Recall, and F1 Score. It also displays a confusion matrix heatmap and a feature coefficient impact chart, explaining the mathematical reasoning behind the predictions.





8. Machine Learning Model & Key Code Snippets

Algorithm — Logistic Regression (Binary Classification): I selected Logistic Regression for this project because the classification task is binary (a student either passes or fails). The model computes the probability of passing by applying the logistic sigmoid function to a linear combination of student features. This algorithm is highly interpretable, meaning we can directly examine its coefficient weights to see how much each feature influences the prediction, avoiding 'black-box' complexity.

To ensure stable model convergence, the input features are standardized using `StandardScaler` to bring all values to a zero mean and unit variance. This prevents features with larger numerical ranges, such as attendance, from drowning out features with smaller ranges, like sleep hours.

Snippet 1 — Synthetic Dataset Generation

Generates 500 student records using NumPy. Output shows dataset shape, pass/fail split, and pass rate.

data_generation.py — Synthetic Dataset Creation

```
1 import numpy as np
2 import pandas as pd
3
4 np.random.seed(42)
5 n_samples = 500
6 hours_study = np.random.uniform(1, 12, n_samples)
7 attendance = np.random.uniform(40, 100, n_samples)
8 prev_marks = np.random.uniform(30, 100, n_samples)
9 assignments_done = np.random.randint(0, 11, n_samples)
10 sleep_hours = np.random.uniform(4, 10, n_samples)
11
12 noise = np.random.normal(0, 5.5, n_samples)
13 final_marks = (0.35 * prev_marks + 2.0 * hours_study + 0.22 * attendance +
14                1.2 * assignments_done + 0.4 * sleep_hours + noise)
15 final_marks = np.clip(final_marks, 0, 100)
16 result = (final_marks >= 50).astype(int)
```

Snippet 2 — Model Training (Logistic Regression)

Scales features with StandardScaler, splits 80/20, fits LogisticRegression. Output confirms successful training.

model_training.py — Logistic Regression Training

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

X = df[['Hours_Study', 'Attendance', 'Previous_Marks', 'Assignments_Done', 'Sleep_Hours']]
y = df['Result']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
scaler = StandardScaler()
X_train_sc = scaler.fit_transform(X_train)
X_test_sc = scaler.transform(X_test)

model = LogisticRegression(max_iter=1000, solver='lbfgs', random_state=42)
model.fit(X_train_sc, y_train)
```

Snippet 3 — Real-time Prediction with Confidence Score

Takes slider inputs, scales them, and calls model.predict() + predict_proba(). Output shows PASS with 91.3% confidence.

prediction.py — Real-time PASS / FAIL Prediction

```
# -- user inputs from Streamlit sliders --
input_df = pd.DataFrame(
    [[hours, attendance, prev_marks, assignments, sleep]],
    columns=['Hours_Study', 'Attendance', 'Previous_Marks', 'Assignments_Done', 'Sleep_Hours']
)
input_scaled = scaler.transform(input_df)
pred = model.predict(input_scaled)[0]
proba = model.predict_proba(input_scaled)[0]

label = 'PASS' if pred == 1 else 'FAIL'
confidence = proba[pred] * 100
```

Snippet 4 — Model Evaluation Metrics

Computes Accuracy, Precision, Recall, F1 Score, and Confusion Matrix on the 100-student test set.

evaluation.py — Model Performance Metrics

```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
from sklearn.metrics import confusion_matrix

y_pred = model.predict(X_test_sc)
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)
cm = confusion_matrix(y_test, y_pred)
```

Snippet 5 — Streamlit Web Interface & Bento Grid UI

Sets up the wide layout, sidebar metadata, responsive CSS animations, and creates columns for the interactive dashboard sliders.

project.py — Web Interface & Dashboard Config

```
import streamlit as st

st.set_page_config(
    page_title='Student Result Prediction System',
    page_icon='■',
    layout='wide'
)

# -- Custom CSS for Hover Animations & Bento Grid Cards --
st.markdown("""
    <style>
    div[data-testid='stVerticalBlockBorder']:has(.study-card-marker) {
        background: linear-gradient(135deg, #f0f9ff 0%, #ffffff 100%) !important;
    }
    div[data-testid='stVerticalBlockBorder']:has(.study-card-marker):hover {
        border-color: #38bdf8 !important;
        transform: translateY(-4px) !important;
    }
    </style>
""", unsafe_allow_html=True)
```

Snippet 6 — Dynamic PDF Prediction Report Generator

Utilises ReportLab to programmatically build and format a downloadable PDF report summarizing individual student results, coefficients, and input parameters.

project.py — Dynamic PDF Report Function

```
1 from reportlab.platypus import SimpleDocTemplate, Paragraph, Spacer, Table, TableStyle
2 from reportlab.lib.styles import getSampleStyleSheet, ParagraphStyle
3 import io
4
5 def generate_pdf_report(hours, attendance, prev_marks, assignments, sleep, prediction, confi
dence):
6     buffer = io.BytesIO()
7     doc = SimpleDocTemplate(buffer, pagesize=letter, rightMargin=54, leftMargin=54, topMargi
n=54, bottomMargin=54)
8     story = []
9
10     styles = getSampleStyleSheet()
11     story.append(Paragraph('Student Result Prediction Report', title_style))
12     # ... adds input profile parameters and evaluation summary ...
13     doc.build(story)
14     buffer.seek(0)
15     return buffer.getvalue()
```

9. Model Performance Metrics

I evaluated the trained model on a test set comprising 100 unseen student records. The resulting evaluation metrics confirm that the classifier is stable and highly capable of identifying at-risk academic profiles:

Metric	Expected Value	What It Means
Accuracy	~88 – 93%	Percentage of overall correct classifications (both Pass and Fail predictions)
Precision	~87 – 92%	Out of all students predicted to PASS, the percentage who actually passed
Recall	~90 – 95%	Out of all students who actually passed, the percentage identified by the model
F1 Score	~88 – 93%	Harmonic mean of Precision and Recall — a balanced measure of overall classifier stability

A high recall score is especially important here: it guarantees that the model catches almost all passing students, thereby minimizing the chance of missing a student who is struggling and needs intervention.

10. How to Run the Project

The project runs locally and has no external dependencies beyond standard Python libraries. To set up and launch:

- **Step 1 — Install libraries:** pip install streamlit scikit-learn pandas numpy matplotlib seaborn reportlab
- **Step 2 — Save script:** Save the complete web application code as project.py in your project directory.
- **Step 3 — Open terminal:** Navigate to the project directory in your terminal or command prompt.
- **Step 4 — Run app:** Start the Streamlit server by running: python -m streamlit run "project.py"
- **Step 5 — View website:** Open your browser and access the live application page at http://localhost:8501.

11. Future Scope

- **Database Integrations:** Connect to real institutional SQL databases to automate data ingestion.
- **Multi-Model Comparison:** Compare Logistic Regression with Random Forest, SVM, or XGBoost algorithms.
- **Cloud Deployment:** Deploy the Streamlit dashboard to Streamlit Cloud for access by teachers anywhere.
- **Authentication:** Build separate login dashboards for students, teachers, and system administrators.

12. Technical Skills Acquired

- **Machine Learning:** Designing classification pipelines, scaling tabular data, and evaluating performance metrics.
- **Frontend Development:** Creating responsive dashboards in Streamlit and writing custom CSS hover animations.
- **Data Engineering:** Preprocessing features, managing DataFrames, and plotting datasets using Pandas and Seaborn.
- **Reporting Automation:** Generating custom PDF reports programmatically using ReportLab flowables.

13. Conclusion

The **Student Result Prediction System** successfully demonstrates the practical application of machine learning to academic challenges. By analyzing key indicators like study hours and attendance, the classifier offers an objective way to flag students who might need academic help, replacing slow and subjective manual observation.

Throughout this project, I completed a full data science workflow, including synthetic data preparation, model training, dashboard visualization, and PDF report creation. Building the frontend in Streamlit allowed for rapid development in pure Python, showing that clean data and simple linear models can provide dependable results when implemented correctly.

14. Project Link

- **Deployed App URL:** https://student-result-prediction-tew9i97r3xr8jqru4p4cu9.streamlit.app